

OnDeviceTraining

Code Reading Guide 3

Line-by-Line Syntax & Explanation — Beginner Edition
Files 20–24: Data Pipeline · Backpropagation · Training Loop · Inference

Mohamed · Master's Thesis · 2026

This guide continues from Guide 2 (which covered files 1–19, Phases 1–8). Guide 3 covers the five file groups that complete the usable training framework: the data pipeline that feeds samples into the model (Dataset, DataLoader, NPYLoader), the backpropagation engine that computes gradients (CalculateGradsSequential), the training-loop orchestration (TrainingBatchDefault, TrainingEpochDefault, TrainingLoopApi), and the inference engine (InferenceApi). Every line of C code is shown with a syntax label on the left and a plain-English explanation on the right. Read Guide 1 first for the phase overview; use this guide to understand the code itself.

Contents

Phase	Topic	Files
Phase 9	Data Pipeline	20 · Dataset.h 21 · DataLoader.h / DataLoader.c 21 · NPYLoader.h / NPYLoader.c
Phase 10	Backpropagation Engine	22 · TrainingLoopApi.h 22 · CalculateGradsSequential.h / .c
Phase 11	Training Orchestration	23 · TrainingBatchDefault 23 · TrainingEpochDefault
Phase 12	Inference Engine	24 · InferenceApi.h / InferenceApi.c
—	Syntax Quick Reference	Last page

Phase 9 — Data Pipeline

Dataset.h · DataLoader.h / DataLoader.c · NPYLoader.h / NPYLoader.c

The data pipeline is the bridge between your .npz files on disk and the tensors the training loop consumes. It works in three layers: **Dataset.h** defines the raw data structures (sample, batch, dataset), **DataLoader.h / DataLoader.c** adds shuffling and batching logic, and **NPYLoader.h / NPYLoader.c** actually reads the binary .npz file format from disk. By the time a tensor reaches the training loop, it has been loaded from disk, parsed, shuffled (if requested), and grouped into a batch of the configured size.

20 · Dataset.h

src/data_loader/include/Dataset.h

Dataset.h is a pure struct-definition header — it contains no functions. It defines four struct types that form the building blocks of the entire data pipeline. Every object that passes between the data loader and the training loop is one of these four types.

<code>#ifndef DATASET_H</code>	Include guard — top. Prevents the compiler from reading this header twice. If DATASET_H is not yet defined, continue; if it is, skip to <code>#endif</code> .
<code>#define DATASET_H</code>	Define the guard symbol. After this line, any second <code>#include</code> of Dataset.h will be skipped entirely.
<code>#include "Tensor.h"</code>	Pull in the <code>tensor_t</code> struct. Every sample and every label is stored in a <code>tensor_t</code> , so the definition of <code>tensor_t</code> must be known before the structs below.
<code>typedef enum { FLOAT_32, INT_32 } dtype_t;</code>	Data-type enumeration. <code>dtype_t</code> has two possible values: <code>FLOAT_32</code> (single-precision floating point, 4 bytes per element) and <code>INT_32</code> (32-bit signed integer). NPYLoader uses this to record what numeric format the .npz file contains. <code>typedef</code> gives the anonymous enum the alias <code>dtype_t</code> so you can write <code>dtype_t x</code> instead of <code>enum {...} x</code> everywhere.
<code>typedef struct sample {</code>	Begin the sample struct. A sample is one training example — one pair of (input, expected-output). <code>typedef + struct</code> together mean you can write <code>sample_t</code> instead of <code>struct sample</code> everywhere.
<code>tensor_t *item;</code>	Pointer to the input tensor. For HAR, this is one 9-channel 128-timestep window of sensor readings, stored as a float32 tensor of shape [9, 128]. An asterisk (*) means pointer — <code>item</code> holds the address where the actual tensor data lives, not a copy.
<code>tensor_t *label;</code>	Pointer to the label tensor. For classification, this is the one-hot encoded class vector (e.g. [0,0,1,0,0,0] for class 2 — WALKING_DOWNSTAIRS). Labels are also <code>tensor_t</code> objects, allowing them to carry quantization and shape information alongside the raw values.
<code>} sample_t;</code>	Close and name the struct. The semicolon is required after every struct definition in C.

<code>typedef struct batch {</code>	Begin the batch struct. A batch groups batchSize many samples together so the model can process them in parallel (vectorised computation, gradient averaging over the batch).
<code>sample_t **samples;</code>	Array of pointers to sample_t objects. Double pointer (**): samples is a pointer to the first element of an array, and each element is itself a pointer to a sample_t. This avoids copying the sample data — only addresses are stored in the array.
<code>size_t size;</code>	The number of samples in this batch. size_t is an unsigned integer type guaranteed to be large enough to hold any object size on the platform (32-bit on ARM Cortex-M, 64-bit on x86-64). Used as the loop bound when iterating over samples.
<code>} batch_t;</code>	Close the batch struct and give it the alias batch_t.
<code>typedef struct tensorArray {</code>	Begin the tensorArray struct. tensorArray is a helper that bundles an array of tensor_t pointers with its length. Used by dataset_t below to store all items and all labels as two separate collections.
<code>tensor_t **array;</code>	Pointer to the first element of an array of tensor_t pointers. The actual tensors live elsewhere in memory; this array holds their addresses.
<code>size_t size;</code>	Number of tensors in the array — i.e. the total number of samples in the split (train, validation, or test).
<code>} tensorArray_t;</code>	Close tensorArray and name it tensorArray_t.
<code>typedef struct dataset {</code>	Begin the top-level dataset struct. A dataset_t pairs all input tensors with all label tensors for one split (e.g. the training set).
<code>tensorArray_t *items;</code>	Pointer to the collection of input tensors. For HAR: 7352 float32 tensors of shape [9, 128] for the training split.
<code>tensorArray_t *labels;</code>	Pointer to the collection of label tensors. Parallel to items: labels->array[i] is the ground-truth class for items->array[i].
<code>} dataset_t;</code>	Close the dataset struct. This struct is the complete in-memory representation of one data split.
<code>#endif // DATASET_H</code>	End of the include guard. Every #ifndef must have a matching #endif.

21 · DataLoader.h / DataLoader.c

src/data_loader/include/DataLoader.h + src/data_loader/DataLoader.c

DataLoader sits on top of Dataset. Its job is to expose a clean interface for the training loop — the training loop only calls `getBatch(loader, i)` and never touches the raw arrays directly. DataLoader also handles optional shuffling (so every epoch sees the data in a different order) and the `dropLast` policy (silently discard the final partial batch if the dataset size is not divisible by `batchSize`).

Header — DataLoader.h

<code>#include</code>	Pull in the <code>bool</code> type. C99 introduced <code>bool</code> (<code>true/false</code>) via this header. It is used for the <code>shuffle</code> and <code>dropLast</code> flags.
<code>typedef struct dataLoader dataLoader_t;</code>	Forward declaration. This line declares that a struct named <code>dataLoader</code> exists and gives it the alias <code>dataLoader_t</code> , without yet showing its fields. This is needed so function pointer typedefs below can refer to <code>dataLoader_t*</code> before the full struct definition appears.
<code>typedef tensor_t *(*transformFn_t)(tensor_t *tensor);</code>	Function pointer type for a transform. A <code>transformFn_t</code> variable holds the address of any function that takes a <code>tensor_t*</code> and returns a <code>tensor_t*</code> . The parentheses around <code>*transformFn_t</code> are required to distinguish a function pointer from a function returning a pointer. Transforms are optional per-sample preprocessing steps (e.g. normalisation).
<code>typedef sample_t *(*getSampleFn_t)(size_t id);</code>	Function pointer type that returns one sample by index. The training code calls <code>loader->getSample(i)</code> and gets back a fully constructed <code>sample_t</code> .
<code>typedef size_t (*getDatasetSizeFn_t)();</code>	Function pointer type for a zero-argument function that returns the total number of samples. Called once during <code>initDataLoader</code> to allocate the index array.
<code>typedef batch_t *(*getBatchFn_t)(dataLoader_t *dataLoader, size_t index);</code>	Function pointer type for fetching one batch. <code>index</code> is the batch number (0, 1, 2 ...). The implementation calls <code>getSample</code> for each slot in the batch, respecting the shuffled index array.
<code>struct dataLoader {</code>	Full struct definition (now that the typedef above already named it <code>dataLoader_t</code>). Each field below stores one piece of the loader's configuration.
<code>getSampleFn_t getSample;</code>	Stored pointer to the user-supplied <code>getSample</code> function. Called by <code>getBatch</code> .
<code>getDatasetSizeFn_t getDatasetSize;</code>	Stored pointer to the size function. Called during <code>init</code> and by the training loop to compute <code>numberOfBatches = datasetSize / batchSize</code> .
<code>uint16_t batchSize;</code>	Number of samples per batch, stored as a 16-bit unsigned integer. <code>uint16_t</code> fits values up to 65535 — more than enough for any realistic batch size.
<code>getBatchFn_t getBatch;</code>	Stored pointer to the <code>getBatch</code> function. The training loop calls this.
<code>transformFn_t transform;</code>	Optional preprocessing applied once after loading (e.g. feature scaling). NULL if not needed.

<code>transformFn_t targetTransform;</code>	Optional transform applied to each label after <code>getBatch</code> . NULL if not needed.
<code>bool shuffle;</code>	If true, the index array is shuffled (random permutation) at init time so each epoch presents samples in a different order.
<code>uint64_t shuffleSeed;</code>	64-bit seed for the shuffle RNG. Using the same seed across runs makes the data order reproducible (important for comparing experiments).
<code>size_t *indices;</code>	Pointer to an array of <code>getDatasetSize()</code> integers. <code>indices[0..N-1]</code> initially holds <code>{0,1,2,...,N-1}</code> . After shuffling, the order is randomised. <code>getBatch</code> uses <code>indices[batchIndex*batchSize + j]</code> instead of a plain integer <code>j</code> to look up the <code>j</code> -th sample in the batch.
<code>bool dropLast;</code>	If true (currently the only supported mode), discard the final batch when <code>datasetSize % batchSize != 0</code> . This ensures every batch has exactly <code>batchSize</code> samples, simplifying gradient accumulation arithmetic.

Implementation — DataLoader.c

<code>#define SOURCE_FILE "DATA_LOADER"</code>	Label this .c file as <code>DATA_LOADER</code> for the <code>PRINT_ERROR</code> macro. When an error is printed, the shell output will show <code>[DATA_LOADER]</code> as the source location.
<code>#include</code>	Fixed-width integer types (<code>uint16_t</code> , <code>size_t</code> , <code>uint64_t</code>).
<code>#include</code>	Standard library: <code>exit()</code> for fatal errors.
<code>#include "Common.h"</code>	<code>PRINT_ERROR</code> macro and debug utilities.
<code>#include "DataLoader.h"</code>	Own header — brings in the <code>dataLoader_t</code> struct definition and function signatures.
<code>#include "RNG.h"</code>	Random number generator. <code>rngSetSeed()</code> and <code>rngShuffleIndices()</code> are called below to shuffle the index array.
<code>void initDataLoader(dataLoader_t *dataLoader, ...)</code>	The only public function. Takes a pointer to an already-allocated <code>dataLoader_t</code> and fills every field. The caller owns the struct (stack or heap).
<code>if (dropLast == false) {</code>	Validation check. <code>dropLast == false</code> (i.e. keep partial batches) is not yet implemented. Calling with <code>dropLast=false</code> immediately exits with an error rather than silently producing wrong batch sizes.
<code>PRINT_ERROR(...); exit(1);</code>	Print an error message and terminate. <code>exit(1)</code> signals abnormal termination to the operating system. On the Pico, <code>exit()</code> triggers a hard fault handler.
<code>dataLoader->getSample = getSample;</code>	Store the function pointer. The arrow operator (<code>-></code>) dereferences the pointer <code>dataLoader</code> and accesses its <code>getSample</code> field. This is equivalent to <code>(*dataLoader).getSample = getSample</code> .

<pre>for (size_t i = 0; i < sizeDataset; ++i) { indices[i] = i; }</pre>	Fill the index array with the identity permutation {0,1,...,N-1}. ++i (pre-increment) and i++ (post-increment) are equivalent here; ++i is a minor style preference to avoid creating an unused temporary.
<pre>if (shuffle) { rngSetSeed(shuffleSeed); rngShuffleIndices(indices, sizeDataset); }</pre>	Conditionally shuffle. rngSetSeed() initialises the xorshift64 generator with shuffleSeed, making the permutation deterministic and reproducible. rngShuffleIndices() performs an in-place Fisher-Yates shuffle: it swaps each element with a randomly chosen element to its right.

21 (continued) · NPYLoader.h / NPYLoader.c

src/data_loader/include/NPYLoader.h + src/data_loader/NPYLoader.c

NPYLoader parses the NumPy binary array format (.npy). A .npy file stores one multi-dimensional array as: a 6-byte magic signature, a 1-byte major version, a 1-byte minor version, a 2-byte (v1) or 4-byte (v2) header length, then a Python-dict-like ASCII header containing the dtype, C/F order, and shape, followed immediately by the raw binary data. NPYLoader reads each field in order and reconstructs the shape metadata and dtype for use by the tensor system.

Header — NPYLoader.h (function declarations)

NPYLoader.h declares six functions, each responsible for one parsing step:

```
FILE *openNPYFile(char *path) – open file, check it exists
void checkMagic(FILE *f) – verify the 6-byte signature
uint32_t readHeaderSize(FILE *f) – read version + header length
void readHeader(char *header, ...) – read the ASCII header string
dtype_t getDTypeFromHeader(char *h) – extract data type (float32/int32)
size_t getNumberOfDimsFromHeader(...) – count dimensions
void getShapeFromHeader(...) – fill the shape_t struct
```

Implementation — NPYLoader.c

<code>FILE *openNPYFile(char *path) {</code>	Open a file in binary read mode. Returns a FILE* — a C standard library file handle. If fopen returns NULL (file does not exist, no permission, etc.), PRINT_ERROR + exit(1) terminates the program.
<code>FILE *f = fopen(path, "rb");</code>	fopen — standard C function to open a file. "rb" = read binary: the file is opened for reading and no newline translation is performed (essential for binary formats where a newline byte 0x0A must be read as-is).
<code>void checkMagic(FILE *f) {</code>	Validate the .npy signature. If this check fails the file is not a valid .npy file and the program exits immediately to avoid reading garbage data.
<code>char magic[7] = {0};</code>	Declare a 7-byte char array, zero-initialised with {0}. The .npy magic is 6 bytes (0x93, 'N','U','M','P','Y'); the 7th byte is reserved for a null terminator so strcmp works correctly.
<code>fread(magic, 1, 6, f);</code>	Read 6 bytes from f into magic. fread arguments: buffer, element size, element count, file pointer. Returns the number of elements successfully read.
<code>if (strcmp(magic, "\x93NUMPY") != 0) {</code>	Compare the 6 bytes to the required magic. \x93 is hexadecimal 0x93 = 147. strcmp returns 0 if the strings are identical; != 0 means mismatch.
<code>uint32_t readHeaderSize(FILE *f) {</code>	Read the format version and header length. Returns the header length as a 32-bit unsigned integer regardless of whether the file is version 1 or 2.
<code>uint8_t major, minor;</code>	Declare two 8-bit variables for the major and minor version numbers. uint8_t is an unsigned byte (0–255). NumPy version 1.0 files are most common; version 2.0 files use a 4-byte header length instead of 2.

<pre>fread(&major;, 1, 1, f); fread(&minor;, 1, 1, f);</pre>	Read one byte each. The & address-of operator passes the address of major/minor so fread knows where to store the byte it reads.
<pre>if (major == 1) { uint16_t hl16; fread(&hl16;, 2, 1, f); header_len = hl16; }</pre>	Version 1: header length is a 2-byte little-endian unsigned integer. Read into uint16_t hl16 (2 bytes), then widen to uint32_t.
<pre>else { fread(&header;_len, 4, 1, f); }</pre>	Version 2: header length is a 4-byte little-endian unsigned integer. Read directly into the uint32_t return value.
<pre>dtype_t getDTypeFromHeader(char *header) {</pre>	Extract the element data type from the ASCII header. The header is a Python dict literal like: {'descr': '<f4', 'fortran_order': False, 'shape': (7352, 9, 128)}.
<pre>char descr[8]; sscanf(strstr(header, " 'descr'"), "'descr': '%7[^]'", descr);</pre>	Parse the descr field. strstr finds the substring 'descr' inside header. sscanf with format '%7[^]' reads up to 7 characters until a single-quote, extracting the type string (e.g. '<f4').
<pre>if (strcmp(s, "<f4") == 0) return FLOAT_32;</pre>	'<f4' means little-endian float32. < = little-endian, f = float, 4 = 4 bytes. FLOAT_32 is the FLOAT_32 member of dtype_t.
<pre>if (strcmp(s, "<i4") == 0) return INT_32;</pre>	'<i4' means little-endian int32. i = signed integer, 4 = 4 bytes.
<pre>size_t getNumberOfDimsFromHeader(char *header) {</pre>	Count how many dimensions the array has. Walks the shape tuple in the header (e.g. '(7352, 9, 128)') and counts how many numbers appear between '(' and ')'.
<pre>char *p = strstr(header, "shape"); p = strchr(p, '(') + 1;</pre>	Find 'shape' in the header, then advance to the character after '('. strchr finds the first '(' from that position; +1 skips the bracket itself.
<pre>while (*p != ')') { ... if (*p >= '0' && *p <= '9') numberOfDims++; ... }</pre>	Walk character by character until ')'. When we see a digit, we have found the start of another dimension integer — increment the counter and skip all digit characters. Skip commas and spaces between numbers.
<pre>void getShapeFromHeader(shape_t *shape, ...) {</pre>	Fill the shape_t struct with dimension sizes. Called after getNumberOfDimsFromHeader has told us how many dimensions to expect. Allocates nothing — the caller passes pre-allocated dims and orderOfDims arrays.
<pre>char *end; size_t value = strtoull(p, &end;, 10);</pre>	Parse one integer from the string. strtoull converts the decimal string at p to a 64-bit unsigned integer. end is set to point to the first character after the number, ready for the next iteration.
<pre>shape->dimensions[i] = value; shape->orderOfDimensions[i] = i;</pre>	Store the dimension size and its canonical order index. orderOfDimensions[i] = i means the dimensions are in row-major (C) order — the default for NumPy arrays.

Phase 10 — Backpropagation Engine

TrainingLoopApi.h · CalculateGradsSequential.h / CalculateGradsSequential.c

Backpropagation is the algorithm that computes gradients — the direction and magnitude in which each weight should change to reduce the loss. CalculateGradsSequential implements the full forward-then-backward pass for a sequential (layer-by-layer) model. It is the most complex file in the framework: it allocates intermediate tensors for every layer's output, runs the forward pass, computes the loss, then walks backwards through the layers calling each layer's backward function to propagate gradients all the way to the first layer.

22 · TrainingLoopApi.h

src/userApi/training_loop/include/TrainingLoopApi.h

TrainingLoopApi.h is the central header for the training loop. It defines all struct types (trainingStats_t, epochStats_t, classificationReport_t) and function pointer typedefs (calculateGradsFn_t, inferenceWithLossFn_t) that tie the backpropagation, inference, and orchestration layers together.

<pre>typedef struct trainingStats { tensor_t *output; float loss; } trainingStats_t;</pre>	Result of one forward+backward pass on a single sample. output holds the model's prediction tensor (class probabilities after Softmax). loss is the scalar loss value for this sample (e.g. cross-entropy).
<pre>typedef struct epochStats { float loss; float accuracy; float precision; float recall; float f1; } epochStats_t;</pre>	Aggregate metrics over a full evaluation epoch. loss: mean cross-entropy. accuracy: fraction of correct predictions. precision, recall, f1: macro-averaged (mean over all classes) — the same metric set reported in the Evci et al. RigL paper.
<pre>typedef struct classificationReport { epochStats_t stats; size_t *confusionMatrix; size_t numClasses; } classificationReport_t;</pre>	Full evaluation result including the confusion matrix. confusionMatrix is a flat array of size numClasses*numClasses: cm[predicted*numClasses + actual] counts how many times class actual was predicted as class predicted. Caller owns and pre-allocates this buffer.
<pre>typedef trainingStats_t (*calculateGradsFn_t)(...);</pre>	Function pointer type for any backpropagation implementation. The training loop is decoupled from the specific backprop strategy — it only knows this type. Passing calculateGradsSequential fills this slot. Custom implementations (e.g. with RigL hooks) can replace it.
<pre>classificationReport_t evaluationEpochWithReport(...);</pre>	Public function that runs the full evaluation epoch and returns a classificationReport_t. Used in main.c after each training epoch and for the final test evaluation. Internally calls inferenceWithLoss for each sample, accumulates the confusion matrix, then computes all metrics.
<pre>trainingRunResult_t trainingRun(...);</pre>	High-level convenience function that runs the complete training loop (all epochs). Not used in the RigL project because RigL needs per-batch hooks (mask application, prune-regrow) that trainingRun does not expose. Instead the project calls trainingBatchDefault directly.

22 (continued) · CalculateGradsSequential.h / CalculateGradsSequential.c

src/userApi/training_loop/calculate_grads/ CalculateGradsSequential.h + .c

CalculateGradsSequential is the core engine. It performs a full forward pass (layers 0 → N-1), computes the loss, then runs the backward pass (layers N-1 → 0). Every intermediate activation tensor is allocated on the heap before the forward pass and freed after the backward pass to keep memory usage bounded.

<pre>trainingStats_t *calculateGradsSequential(layer_t **model, size_t modelSize, ...)</pre>	Main entry point. Takes the model array, its size, a loss configuration (which loss function, which reduction), one input tensor, and one label tensor. Returns a heap-allocated trainingStats_t containing the prediction and loss.
<pre>tensor_t *layerOutputs[modelSize + 1];</pre>	Variable-length array (VLA) of tensor pointers — one per layer plus the input. layerOutputs[0] = input (the raw sensor data). After the forward pass, layerOutputs[i] holds the output activation of layer i-1. VLAs are allocated on the call stack, so the array of pointers is stack memory; the actual tensor data they point to is heap memory (via reserveMemory).
<pre>layerOutputs[0] = input;</pre>	The first 'activation' is simply the input tensor — no layer has processed it yet. The forward loop starts with this as its input.
<pre>setDropoutLayersTraining(model, modelSize, true);</pre>	Enable dropout for training. Dropout randomly zeroes some activations during training to reduce overfitting, but must be disabled during evaluation so every neuron contributes deterministically. This function sets a training flag in each DROPOUT layer config.
<pre>initLayerOutputs(layerOutputs, model, modelSize);</pre>	Allocate output tensors for every layer. This calls calcOutputShape for each layer to determine the output dimensions, then reserveMemory to allocate the data buffer and associated shape/quantization structs.
<pre>for (size_t i = 0; i < modelSize; i++) { ... forward(...); }</pre>	Forward pass loop. For each layer i, looks up its forward function in the layerFunctions vtable (layerFunctions[type].forward), then calls it with layerOutputs[i] as input and layerOutputs[i+1] as the output buffer.
<pre>float loss = lossFns.forward(layerOut puts[modelSize], label, forwardReduction);</pre>	Compute the scalar loss. lossFns is a row of the lossFunctions dispatch table selected by lossConfig.funcType (e.g. CROSS_ENTROPY). The forward function computes cross-entropy between the model's class-probability output and the one-hot label vector.
<pre>if (lossConfig.funcType == CROSS_ENTROPY) { backwardIndex -= 1; }</pre>	Special case for cross-entropy + softmax. The cross-entropy backward pass is fused with the Softmax backward pass (their combined gradient simplifies to prediction - label). Skipping the Softmax backward layer avoids computing the Jacobian of Softmax twice.
<pre>tensor_t gradNext; initGradTensor(&gradNext;, layerOutputs[modelSize]);</pre>	Allocate the initial gradient tensor. At the start of backpropagation (the output end), gradNext holds dLoss/dOutput — how the loss changes with each element of the final layer's output. Its shape matches the output.
<pre>lossFns.backward(layerOutputs[modelSi ze], label, &gradNext);</pre>	Compute the loss gradient with respect to the model output. For cross-entropy, this is (softmax_output - one_hot_label) — a simple element-wise subtraction.
<pre>for (int i = (int)backwardIndex; i >= 0; i--) { ... backward(...); }</pre>	Backward pass loop — runs in reverse order. For each layer i, calls its backward function which: (a) accumulates gradients into the layer's weight gradient tensors (used by the optimizer), and (b) computes gradCurr — the gradient to pass further back.

<pre>backward(model[i], layerOutputs[i], &gradNext;, &gradCurr;);</pre>	Each layer's backward function signature: layer config (holds weight tensors), the layer's saved input (needed to compute weight gradients), the gradient coming from the next layer (gradNext), and a buffer for the gradient to pass to the previous layer (gradCurr). After the call, gradCurr becomes gradNext for the next iteration.
<pre>deInitGradTensor(&gradNext;); gradNext = gradCurr;</pre>	Free the previous gradNext and advance: gradNext = gradCurr. This is the 'shift' step — each iteration processes one layer and passes its gradient one step further back.
<pre>deInitLayerOutputs(layerOutputs, modelSize);</pre>	Free all intermediate activation tensors allocated during initLayerOutputs. They were only needed for the backward pass (to compute weight gradients); once backprop is done, they can be released.

Phase 11 — Training Orchestration

TrainingBatchDefault.h / .c · TrainingEpochDefault.h / .c

The orchestration layer calls `CalculateGradsSequential` in the right order and feeds gradients to the optimizer. It is split into two levels: **TrainingBatchDefault** processes one batch (iterates over the batch's samples, accumulates loss), and **TrainingEpochDefault** processes one full epoch (iterates over all batches, steps the optimizer after each).

23 · TrainingBatchDefault.h / TrainingBatchDefault.c

src/userApi/training_loop/training_batch/TrainingBatchDefault.h + .c

`TrainingBatchDefault` is intentionally minimal — its only job is to iterate over the samples in a batch, call the gradient-computation function for each, accumulate the total loss, and return the mean. The optimizer step is NOT called here; it happens in `TrainingEpochDefault` after the batch is complete.

<pre>float trainingBatchDefault(layer_t **model, size_t modelSize,</pre>	Returns the loss for this batch as a float. <code>model</code> and <code>modelSize</code> describe the network. <code>lossConfig</code> selects the loss function. <code>batch</code> holds the samples. <code>calculateGradsFn</code> is a function pointer — passing <code>calculateGradsSequential</code> here decouples the batch function from any specific backprop implementation.
<pre>for (size_t i = 0; i < batch->size; i++) {</pre>	Iterate over every sample in the batch. <code>batch->size</code> equals <code>batchSize</code> (the <code>dropLast</code> policy guarantees no partial batches).
<pre>trainingStats_t *stats = calculateGradsFn(model, modelSize, lossConfig,</pre>	Call the backpropagation function for this single sample. <code>calculateGradsFn</code> is the function pointer passed by the caller — in normal use this is <code>calculateGradsSequential</code> , but for <code>RigL</code> the project calls <code>trainingBatchDefault</code> from its own custom epoch loop.
<pre>forwardReduction, batch->samples[i]->item, batch->samples[i]->label);</pre>	Pass the <code>i</code> -th sample's input tensor and its label tensor. <code>->item</code> is the input (sensor readings), <code>->label</code> is the one-hot target.
<pre>totalLoss += stats->loss;</pre>	Accumulate the scalar loss. After the loop, <code>totalLoss</code> is the sum of per-sample losses across the batch.
<pre>freeTrainingStats(stats);</pre>	Release the <code>trainingStats_t</code> returned by <code>calculateGradsFn</code> . This frees the output prediction tensor and the struct itself.
<pre>freeSample(batch->samples[i]);</pre>	Release the sample's memory. Each <code>getBatch</code> call allocates <code>sample_t</code> structs and their tensors; <code>freeSample</code> releases them. The underlying dataset arrays are NOT freed — only the per-batch wrappers.
<pre>if (forwardReduction == REDUCTION_MEAN) { return totalLoss / (float)batch->size; }</pre>	If loss was averaged per-sample inside <code>calculateGradsSequential</code> (<code>REDUCTION_MEAN</code>), return the mean over the batch. The <code>(float)</code> cast prevents integer division. If <code>REDUCTION_SUM</code> , return the raw sum.

23 (continued) · TrainingEpochDefault.h / TrainingEpochDefault.c

src/userApi/training_loop/training_epoch/TrainingEpochDefault.h + .c

`TrainingEpochDefault` is the outer loop — it iterates over all batches in the training `DataLoader` for one full pass through the training set. After each batch, it applies the mean-scale correction (if `REDUCTION_MEAN`), steps the optimizer (updates weights), and zeroes the gradient accumulators. At the end it returns the mean epoch

loss.

<pre>float trainingEpochDefault(layer_t **model, size_t modelSize, lossConfig_t lossConfig,</pre>	Returns the mean training loss for this epoch. All arguments are passed straight through to trainingBatchDefault, plus the optimizer.
<pre>size_t numberOfBatches = datasetSize / dataLoader->batchSize;</pre>	Compute how many full batches fit in the dataset. Integer division silently discards the remainder (the dropLast guarantee means no partial batches).
<pre>optimizerFunctions_t optimFns = optimizerFunctions[optimizer->type];</pre>	Fetch the optimizer's vtable row. optimizerFunctions is a dispatch table (defined in Optimizer.c) that maps optimizer type (e.g. SGD) to its step, zero, and scale functions. Caching the row in a local variable avoids repeated table lookups inside the loop.
<pre>batch_t *batch = dataLoader->getBatch(dataLoader, i);</pre>	Fetch the i-th batch. getBatch uses the shuffled index array to pick batchSize samples in the shuffled order for this epoch.
<pre>tensor_t *labelRef = batch->samples[0]->label;</pre>	Save a pointer to the first sample's label BEFORE trainingBatchDefault consumes the batch. freeSample (called inside trainingBatchDefault) only frees the sample_t wrapper — the underlying label tensor in the dataset array remains alive, so labelRef remains valid for computeMeanScale below.
<pre>totalLoss += trainingBatchDefault(model, modelSize, lossConfig, batch, ...);</pre>	Process the batch: forward pass for all samples, accumulate gradients, return the batch loss. Note: the optimizer step happens AFTER this call, so all samples in the batch contribute to the gradients before any weight update.
<pre>if (lossConfig.backwardReduction == REDUCTION_MEAN) {</pre>	If gradient reduction is MEAN: the gradients accumulated across the batch are per-sample averages, but the optimizer needs the batch-mean gradient. Apply a correction factor so the weight update magnitude is independent of batch size.
<pre>float meanScale = lossFunctions[...]. computeMeanScale(batch->size, labelRef);</pre>	Compute the scaling factor. For cross-entropy with one-hot labels, computeMeanScale returns $1/(\text{batch->size} * \text{NUM_CLASSES})$, normalising the gradient by both the batch size and the label vector width.
<pre>scaleOptimizerGradients(optimizer, meanScale);</pre>	Multiply every parameter's gradient accumulator by meanScale. This is applied once per batch, after all samples have contributed.
<pre>optimFns.step(optimizer);</pre>	Run the optimizer update step. For SGD with momentum: $\text{velocity}_i = \text{momentum} * \text{velocity}_i - \text{lr} * \text{gradient}_i$; $\text{weight}_i += \text{velocity}_i$. This modifies the weight tensors in place.
<pre>optimFns.zero(optimizer);</pre>	Reset all gradient accumulators to zero. Must be called after every optimizer step, before the next batch's gradients start accumulating. Forgetting this would sum gradients across batches (unintended).
<pre>freeBatch(batch);</pre>	Free the batch struct and the sample_t wrappers. The actual tensor data in the dataset arrays is NOT freed — only the getBatch-allocated wrappers.

```
return totalLoss /  
(float)numberOfBatches;
```

Return the mean epoch loss: sum of all batch losses divided by the number of batches. This is the number printed as 'Training Loss: X.XXX'.

Phase 12 — Inference Engine

InferenceApi.h / InferenceApi.c

InferenceApi runs the model in forward-only mode — no gradient computation, no backward pass. It is used during evaluation (to measure accuracy on the validation/test set) and for deployment on the Pico (to classify new sensor windows). Unlike CalculateGradsSequential which allocates a separate output tensor for every layer simultaneously, InferenceApi uses a ping-pong buffer strategy: it keeps only two tensors alive at any time (the current layer's input and the current layer's output), dramatically reducing peak memory use.

24 · InferenceApi.h

src/userApi/include/InferenceApi.h

<pre>typedef struct inferenceStats { tensor_t *output; float loss; } inferenceStats_t;</pre>	Result of one inference call with loss computation. output holds the model's prediction (class probabilities). loss is the cross-entropy against the ground-truth label. Analogous to trainingStats_t but without gradient information.
<pre>tensor_t *inference(layer_t **model, size_t numberOfLayers, tensor_t *input);</pre>	Run a single forward pass. Returns a heap-allocated tensor_t containing the model's output (class probabilities after Softmax). The caller is responsible for freeing this tensor.
<pre>tensor_t **inferenceBatched(layer_t **model, size_t numberOfLayers, batch_t *batch);</pre>	Run inference on a full batch. Returns an array of tensor_t pointers — one per sample. Used when evaluating multiple samples at once. The caller frees both the array and each tensor.
<pre>inferenceStats_t *inferenceWithLoss(..., lossFuncType_t funcType, reduction_t fwd);</pre>	Run inference AND compute the loss against the supplied label. This is what the evaluation loop uses: it needs both the prediction (to check correctness) and the loss (to track test loss).

24 (continued) · InferenceApi.c — the ping-pong buffer pattern

The key architectural decision in InferenceApi.c is the ping-pong buffer: instead of pre-allocating N+1 tensors (one per layer), it allocates just two — outputNext (the current input to the layer) and outputCurr (the current layer's output). After each layer, outputNext is freed and replaced by outputCurr. This keeps memory proportional to the single largest layer output rather than the sum of all layer outputs.

<pre>tensor_t *inference(layer_t **model, size_t numberOfLayers, tensor_t *input) {</pre>	Entry point for single-sample inference. Returns a tensor_t* that the caller owns and must eventually free.
<pre> tensor_t outputNext; initBufferInput(input, &outputNext);</pre>	Create a copy of the input tensor in a local stack variable. initBufferInput allocates heap memory for the data buffer and shape/quant structs, then copies the input values. Using a local stack variable for the tensor_t struct (not a pointer) avoids one heap allocation.
<pre> for (size_t i = 0; i < numberOfLayers; i++) {</pre>	Iterate over every layer in the model, left to right.
<pre> tensor_t outputCurr;</pre>	Stack variable for the current layer's output tensor. Like outputNext, the tensor_t header is on the stack; only its data buffer is on the heap.

<code>initBufferOutput(&outputCurr;, currentLayer, outputNext.shape, ...);</code>	Allocate the output buffer. Calls <code>calcOutputShape</code> to determine the output dimensions (e.g. after a <code>MaxPool</code> with stride 2, the length halves), then <code>reserveMemory</code> for the data.
<code>forward(currentLayer, &outputNext;, &outputCurr;);</code>	Run the layer's forward computation: reads from <code>outputNext</code> , writes to <code>outputCurr</code> . The specific function called depends on <code>currentLayerType</code> (via the <code>layerFunctions</code> vtable).
<code>deInitBuffer(&outputNext;); outputNext = outputCurr;</code>	The ping-pong step: free <code>outputNext</code> (its data is no longer needed), then copy the <code>outputCurr</code> struct into <code>outputNext</code> for the next iteration. This is a struct assignment — it copies the pointer values inside the struct, not the pointed-to data. The heap memory now belongs to <code>outputNext</code> .
<code>tensor_t *output = getTensorLike(&outputNext;);</code>	Allocate the final return tensor with the same shape and quantization as the last layer's output.
<code>convertTensor(&outputNext;, output);</code>	Copy (and optionally type-convert) the last activation into <code>output</code> . <code>convertTensor</code> handles quantization type differences.
<code>deInitBuffer(&outputNext;); return output;</code>	Free the last ping-pong buffer, then return the allocated output tensor. The caller owns this tensor and must call <code>freeTensor</code> when done.
<code>inferenceStats_t *inferenceWithLoss(... tensor_t *label, lossFuncType_t funcType ...) {</code>	Same forward pass as <code>inference()</code> , plus a loss computation at the end. Used by the evaluation loop to compute both test accuracy and test loss.
<code>float loss = lossFns.forward(&outputNext;, label, forwardReduction);</code>	Compute the scalar loss between the model's output and the label. For cross-entropy: $\text{loss} = -\sum(\text{label}_i * \log(\text{output}_i))$. The result is stored in <code>inferenceStats->loss</code> .

Syntax Quick Reference — Guide 3 Additions

This table collects every new C syntax construct introduced in Guide 3 that did not appear in Guide 2.

Syntax	Full name	What it does
<code>FILE *f = fopen(p, mode)</code>	File open	Opens the file at path <code>p</code> . mode <code>'rb'</code> = read binary. Returns <code>NULL</code> on failure.
<code>fread(buf, size, n, f)</code>	File read	Reads <code>n</code> items of size bytes each from file <code>f</code> into buffer <code>buf</code> .
<code>strcmp(a, b)</code>	String compare	Returns 0 if strings <code>a</code> and <code>b</code> are identical, nonzero otherwise.
<code>sscanf(str, fmt, ...)</code>	String scan	Parses <code>str</code> using format string <code>fmt</code> , stores results in the following pointers.
<code>strstr(hay, needle)</code>	String search	Returns a pointer to the first occurrence of <code>needle</code> inside <code>hay</code> , or <code>NULL</code> .
<code>strtoull(p, &end, base)</code>	String to unsigned long long	Converts the decimal number at <code>p</code> to <code>size_t</code> . Sets <code>end</code> to the first non-digit.
<code>tensor_t x; (stack)</code>	Stack-allocated struct	The struct header lives on the call stack. The data it points to may still be heap.
<code>tensor_t *x; (heap)</code>	Heap-allocated struct	<code>reserveMemory</code> allocates the struct on the heap. Must be explicitly freed.
<code>outputNext = outputCurr;</code>	Struct assignment	Copies all fields of the struct (pointer values, not pointed-to data).
<code>for (int i=(int)N-1; i>=0; i--)</code>	Reverse loop	Walks an array backwards. The cast <code>(int)</code> is needed because <code>size_t</code> is unsigned and <code>i>=0</code> would never be false for <code>size_t</code> .
<code>vtable[type].fn(args)</code>	Dispatch table call	Selects the correct function for this layer type and calls it. C-style polymorphism without <code>virtual</code> keyword.
<code>fn_ptr(args)</code>	Function pointer call	Calls the function whose address is stored in <code>fn_ptr</code> . Used for <code>calculateGradsFn</code> .

Next step — Guide 4: Conv1d / Linear layer internals, Pool layers, Optimizer internals, Serialization (Serialize / Deserialize), and the HAR example main program.

Mohamed (matef517@gmail.com) · OnDevice HAR FQT+RigL · 2026